

LeetCode 1775: Equal Sum Arrays With Minimum Number of Operations

Company Tag: American Express

Problem Intuition & Algorithm

Let there be two arrays: one with a **lower sum** ($lsum$) and one with a **higher sum** ($hsum$).

- The array with the lower sum can **increase** its elements to reduce the difference.
- The array with the higher sum can **decrease** its elements to reduce the difference.

Step 1 (S1): Compute both sums, $lsum$ and $hsum$.

Step 2 (S2): Calculate Potential Gains

For the smaller array (elements can increase up to 6):

Example: $1 \rightarrow 6$ (Gain = 5)

Formula: $gain = 6 - x$

For the larger array (elements can decrease down to 1):

Example: $6 \rightarrow 1$ (Gain = 5)

Formula: $gain = x - 1$

Step 3 (S3): Create a gain array. Push all the computed gains into it and sort them in **descending order** (largest gain first).

Step 4 (S4): Keep picking the maximum available gain until the remaining difference (rem) is ≤ 0 .

Time Complexity: $O((n + m) \log(n + m))$

C++ Implementation

```
class Solution {
public:
    int minOperations(vector& nums1, vector& nums2) {

        int lsum = accumulate(nums1.begin(), nums1.end(), 0);
        int hsum = accumulate(nums2.begin(), nums2.end(), 0);

        if (hsum < lsum) {
            swap(lsum, hsum);
            swap(nums1, nums2);
        }

        vector gains;
        for (int n : nums1) {
            gains.push_back(6 - n);
        }

        for (int m : nums2) {
            gains.push_back(m - 1);
        }

        sort(gains.begin(), gains.end(), greater());

        int ans = 0;
        int rem = hsum - lsum;

        for (int gain : gains) {
            if (rem <= 0) {
                return ans;
            }

            rem -= gain;
            ans++;
        }

        if (rem > 0) return -1;
        else return ans;
    }
};
```

